distribute_stack.sh

Field	Value
Revision	2
Last modified	2026-06-22T20:30:00Z
Status	active
§11.4.18	In-source doc block present in `scripts/distribute_stack.sh`

Overview

Distributes the helix-layer **HelixTrack container stack** onto a remote distribution host using **SSH + remote rootless `podman compose`** (§11.4.161 — no sudo, no rootful docker on the remote). The mechanism is:

1. **Probe** each candidate host for SSH reachability AND rootless podman presence (`podman compose` plugin, falling back to `podman-compose`).
2. **rsync** a self-contained deploy bundle to `~/helix-dist/` on the remote, preserving the relative layout the compose file expects.
3. Run **`podman compose -f compose.helixtrack.yml up -d` ON the remote** over SSH (rootless).
4. **Health-check** the deployed stack remotely (`curl http://localhost:8080`).
5. Capture the real `podman compose ps` / health output.

The first reachable host **with rootless podman** wins. A host that is SSH- reachable but has no rootless podman is **skipped honestly** (exit 0, no bluff per §11.4.6) — never a fake success **unless** the operator has explicitly authorised the docker fallback for that host (see below).

§11.4.161 operator-authorized docker fallback (hosts without rootless podman)

§11.4.161 mandates **rootless podman** for all containerized workloads; Docker in rootful mode / sudo / root escalation is FORBIDDEN **unless the target platform has no rootless option AND that constraint is documented per §11.4.112**. Some distribution hosts (e.g. `amber`) have **no rootless podman installed** — only `docker`. For those hosts the script supports an **operator-authorized docker fallback**, gated so it is NEVER silent and NEVER the default:

- The fallback runs `docker compose` (or `docker-compose`) on the remote **only** when the operator explicitly opts in via **`HELIX_ALLOW_DOCKER_FALLBACK=1`** AND the host has been confirmed to have no rootless-podman option.

- Without that flag a podman-less host is **SKIPPED honestly** (the §11.4.161 default — rootless-or-nothing); the docker path is an authorized exception, not a convenience.
- The authorization + the "no rootless option on this host" justification are the §11.4.112 documented-constraint requirement — the operator records *why* the host cannot run rootless podman (e.g. "amber: docker-only, rootless podman not yet installed; tracked operator action item to install it"), so the exception is documented, narrow, and reversible (install rootless podman → drop the flag).
- Preferred remediation stays **install rootless podman on the host** (then the fallback is unnecessary); the docker fallback is the bridge until that is done, not the destination.

```
# §11.4.161 docker fallback — operator-authorized, explicit opt-in ONLY.
# Use when the target host (e.g. amber) has docker but no rootless podman,
# and the operator accepts the documented §11.4.112 constraint.
HELIX_ALLOW_DOCKER_FALLBACK=1 \
  bash scripts/distribute_stack.sh --host amber.local --user milosvasic
```

Honest boundary (§11.4.6): the docker fallback is a documented exception for a host with no rootless option — it does NOT relax §11.4.161 for hosts that COULD run rootless podman; on such a host the only correct path is rootless podman.

This lives in the **helix layer**, NOT inside the generic `vasic-digital/containers` submodule (§11.4.28 — no project hostnames or project-specific deploy mechanism inside the reusable submodule).

Prerequisites

- `ssh` + `rsync` on the host running the script.
- Passwordless SSH key auth to the distribution host for the configured user.
- **Rootless podman** on the remote (§11.4.161): `podman` + either the `podman compose` plugin or `podman-compose`.
- The compose file `containers/compose.helixtrack.yml` present locally.
- The build context — `helix_track/core/Application/` with a `Dockerfile`. In this checkout it is resolved from the **sibling** repo `/Volumes/T7/Projects/helix_track` (override via `HELIX_TRACK_SRC`). Use `--no-build-context` if the remote already has the context / a prebuilt image.

Usage

```
# Probe + print the exact rsync/compose commands that WOULD run — no deploy:
bash scripts/distribute_stack.sh --dry-run

# Real deploy to the first reachable podman-ready host:
bash scripts/distribute_stack.sh

# Target a single host explicitly:
```

```bash
bash scripts/distribute_stack.sh --host thinker.local --user
       milosvasic

# Skip rsyncing the build context (remote already has it / a
       prebuilt image):
bash scripts/distribute_stack.sh --no-build-context

# Tear the stack down on the remote:
bash scripts/distribute_stack.sh --down
```

Environment overrides

Var	Default	Meaning
`HELIXTRACK_REMOTE_HOST`	`thinker.local amber.local`	Space-separated candidate hosts.
`HELIXTRACK_REMOTE_USER`	`milosvasic`	SSH user.
`HELIX_DIST_REMOTE_DIR`	`~/helix-dist`	Remote bundle directory.
`HELIX_TRACK_SRC`	autodetect sibling	Build-context source repo root.

Distribution-host status (operator decision 2026-06-22)

Host	SSH	Rootless podman	Docker	Status
`thinker.local`	key works (milosvasic)	podman 4.9.3 + podman-compose 1.0.6 + `podman compose` plugin	—	**LIVE** distribution target (rootless podman — preferred path)
`amber.local`	key works (onboarded 2026-06-22)	**NOT installed**	present	**DOCKER-FALLBACK target** — `HELIX_ALLOW_DOCKER_FALLBACK=1` (§11.4.161 operator-authorized exception, §11.4.112 documented constraint: rootless podman not yet installed; tracked operator action item to install it). Without the flag → SKIPPED honestly.
`nezha.local`	—	—	—	**NOT** a distribution target (read/import + emulator host only)

Integration with the per-commit flow

The per-commit HelixTrack sync (`scripts/commit_all.sh::_helixtrack_ensure_running` and `scripts/git_hooks/pre-commit`) stays **lightweight**: it only needs a reachable HelixTrack endpoint for the workable-items sync and does **NOT** fire a heavy remote container deploy on every commit. This script is invoked from those paths **only** when the operator opts in with `HELIX_DISTRIBUTE_ON_COMMIT=1` (default OFF). Routine commits therefore never trigger a remote deploy; run the script explicitly when distribution is actually wanted.

Edge cases

- **No host has rootless podman** → SKIP (exit 0), no deploy, no bluff.
- **SSH reachable but podman absent** (amber today) → that host skipped, next candidate tried.
- **Build context missing locally** (and not `--no-build-context`) → SKIP with a clear message (the remote build would otherwise fail).
- **Deploy succeeds but health-check fails within 60s** → exit 1 (a genuine failure, reported — not silently skipped).

Internal behaviour

`--dry-run` performs the real SSH + podman-presence probe (so the printed plan reflects the actually-selected host and compose front-end) but stops before any rsync or `podman compose up`. The real deploy rsyncs the compose file + build context + `helix_track/spaces` volume data into the remote bundle, then runs the rootless compose and polls the health endpoint.

Related scripts

- `scripts/commit_all.sh` — per-commit lightweight HelixTrack sync; opt-in delegation to this script via `HELIX_DISTRIBUTE_ON_COMMIT=1`.
- `scripts/git_hooks/pre-commit` — same lightweight sync at hook time.
- `scripts/boot_android_emulator.sh` — remote emulator boot on `nezha.local` (read/import host, NOT a distribution target).
- `containers/compose.helixtrack.yml` — the compose stack deployed.

Last verified: 2026-06-22 (dry-run probe against thinker.local: SSH OK, `podman compose` selected; amber.local onboarded — SSH key installed + docker present, §11.4.161 operator-authorized docker fallback available via `HELIX_ALLOW_DOCKER_FALLBACK=1`, honestly skipped without the flag).